Atividade Prática - Automação de Testes Integrada (Missão Marte: ECLSS)

Visão Geral e Objetivos

Esta atividade prática visa consolidar a aplicação de técnicas e ferramentas de garantia de qualidade de software em um projeto em Python. Os alunos assumirão o papel de engenheiros de testes da agência espacial encarregados de validar o **Environmental Control and Life Support System (ECLSS)** do Habitat de Marte.

O objetivo é desenvolver uma suíte de testes automatizados utilizando o **pytest** como framework central para testes unitários e de integração, isolamento com Mocks, automação de interface Web com Selenium WebDriver em modo **headless**, cenários BDD com sintaxe Gherkin e execução contínua via GitHub Actions.

Escopo do Sistema Sob Teste (SUT)

O projeto consiste no módulo de controle e monitoramento do habitat espacial composto por:

1. `GerenciadorRecursosMarte`: Regra de negócio responsável pela validação de estoque de oxigênio e água, cálculo de consumo por astronauta/dia, cálculo de taxa de emergência e aplicação de cotas de racionamento.
2. `ServicoTelemetriaEspacial`: Dependência externa responsável por enviar relatórios de status e disparar alertas de crise para a estação na Terra.
3. `PainelControleWeb`: Aplicação Web simples (`painel.html`) para consulta de recursos e solicitação de cota extra de oxigênio pelos astronautas.

Requisitos Obrigatórios de Teste

Requisito 1: Testes Unitários e Integração com Pytest

- **Estruturação com Fixtures:**
  - Definir ao menos uma fixture global/módulo no arquivo `conftest.py` para carregamento dos parâmetros iniciais do habitat.
  - Utilizar fixtures de escopo `function` para prover instâncias limpas do `GerenciadorRecursosMarte`.
  - Aplicar o padrão de preparação e limpeza utilizando a instrução `yield`.
- **Testes Orientados a Dados (Parametrização):**

    - Utilizar `@pytest.mark.parametrize` para testar múltiplos cenários de consumo diário, variação do número de astronautas e taxas de emergência em uma única função de teste.
- **Validação de Exceções:**
    - Garantir a validação de regras de falha e exceções customizadas (ex: `OxigenioInsuficienteError` ou `NivelEmergenciaInvalidoError`) utilizando `with pytest.raises(...)`.

Requisito 2: Isolamento com Dublês e Mocks (`pytest-mock`)

- **Isolamento de Dependências:**
    - Isolar as chamadas de rede e comunicação externa do `ServicoTelemetriaEspacial` durante a execução dos testes unitários do `GerenciadorRecursosMarte`.
- **Verificação de Comportamento:**
    - Utilizar `mocker.patch` ou `MagicMock` para simular as respostas da estação terrestre.
    - Validar a interação e disparo de alertas de emergência garantindo que os métodos foram chamados com os argumentos corretos (`assert_called_once_with`).

Requisito 3: Automação Web End-to-End (E2E) com Selenium WebDriver & Pytest

- **Gerenciamento do Ciclo de Vida do Browser:**
    - Criar uma fixture no `pytest` para inicializar e encerrar o driver (`driver.quit()`) configurado em modo **headless**.
- **Interação e Sincronização:**
    - Automatizar o fluxo de preenchimento do formulário de solicitação de oxigênio no `painel.html`.
    - Empregar **Esperas Explícitas** (`WebDriverWait` com `expected_conditions`) para aguardar a atualização dos níveis no DOM, proibindo o uso de pausas estáticas (`time.sleep()`).
- **Auditoria e Evidências:**
    - Implementar a captura automática da tela em caso de falha de validação utilizando `driver.save_screenshot()`.

Requisito 4: BDD (Behavior-Driven Development) com Pytest-BDD

- **Especificação de Cenários:**

    - Escrever a especificação em linguagem Gherkin no arquivo `.feature` descrevendo cenários de solicitação de suprimentos em condições normais e de crise (`Given`, `When`, `Then`).
- **Mapeamento no Pytest:**
    - Mapear e implementar os passos (steps) correspondentes em Python utilizando os decoradores `@given`, `@when` e `@then` da biblioteca `pytest-bdd`.

Requisito 5: Integração Contínua (CI) com GitHub Actions

- **Pipeline Automatizado (`.github/workflows/pipeline.yml`):**
    - Configurar o disparo automático em eventos de `push` e `pull_request`.
    - Incluir etapas para checkout do código (`actions/checkout@v4`), setup do ambiente Python (`actions/setup-python@v5`) e instalação das dependências.
    - Executar toda a suíte via `pytest` exibindo o relatório de cobertura de código no console (`pytest-cov`).

Estrutura Recomendada do Repositório

```
projeto-missao-marte/
├── .github/
│   └── workflows/
│       └── pipeline.yml
├── src/
│   ├── __init__.py
│   ├── gerenciador_recursos.py
│   └── servico_telemetria.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_gerenciador.py
│   ├── test_selenium_e2e.py
│   ├── test_bdd.py
│   └── features/
│       └── solicitacao_oxigenio.feature
├── web/
│   └── painel.html
├── requirements.txt
└── README.md
```

Rubrica de Avaliação

Critério	Peso	Descrição
Suíte Pytest (Fixtures, Parametrização e Exceções)	35%	Uso completo de fixtures, parametrização de cenários e captura correta de exceções com `pytest.raises`.
Isolamento com Mocks/Stubs	20%	Substituição correta de dependências de telemetria e asserção do comportamento de chamadas de alerta.
Automação Web com Selenium & Pytest	20%	Validação de fluxo Web no painel com esperas explícitas, fixture para o WebDriver e captura de evidências visual.
Especificação e Passos BDD	15%	Escrita clara em Gherkin e mapeamento correto dos passos com `pytest-bdd`.

Gerado em: 26/10/2026

Critério	Peso	Descrição
Integração Contínua (CI) e Relatório de Cobertura	10%	Workflow funcional no GitHub Actions gerando relatório de cobertura (`pytest-cov`).

Instruções de Entrega

1. **Link do Repositório:** Submeta a URL do repositório público do GitHub contendo o código-fonte desenvolvido, a suíte de testes e o arquivo do pipeline (`.github/workflows/pipeline.yml`).
2. **Execução em CI Obrigatória no "Verde":** É requisito indispensável que a suíte completa de testes automatizados (Pytest, Mocks, Selenium Headless e BDD) execute integralmente e seja aprovada com sucesso (status verde / passed) na pipeline de integração contínua do GitHub Actions.
3. **Badge de Build no README:** O arquivo `README.md` deve conter a badge de status do workflow do GitHub Actions no topo do documento, comprovando visualmente a execução bem-sucedida da suíte.
4. **Documentação de Execução Local:** Inclua no `README.md` o passo a passo para clonar o repositório, instalar os requisitos (`pip install -r requirements.txt`) e executar a suíte de testes localmente via terminal com o comando `pytest`.